

자료구조

2분반

HW2

보고서

담당교수님 : 장형수 교수님

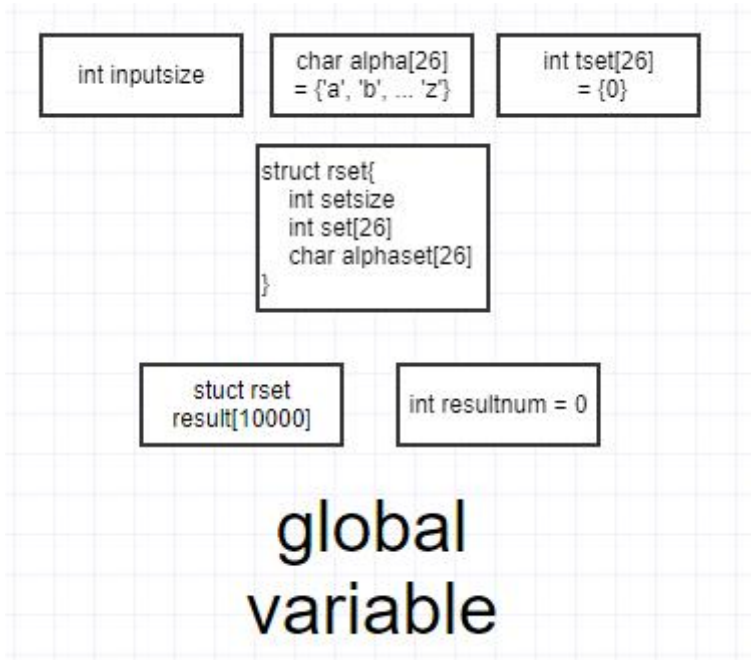
학과 : 컴퓨터공학과

학번: 20211562

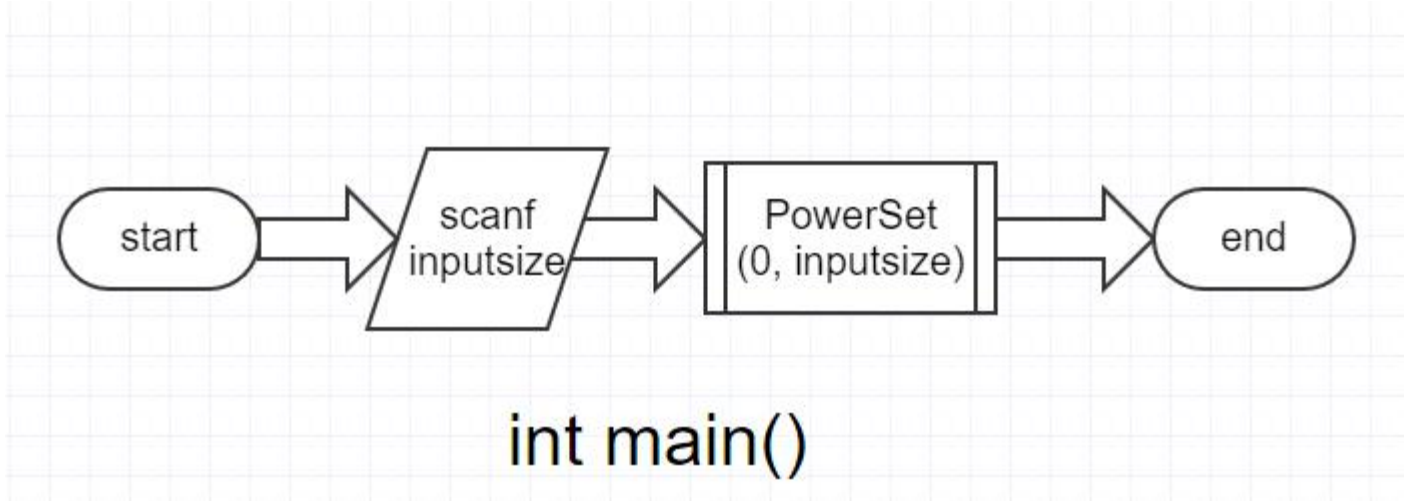
이름: 이동협

문제1

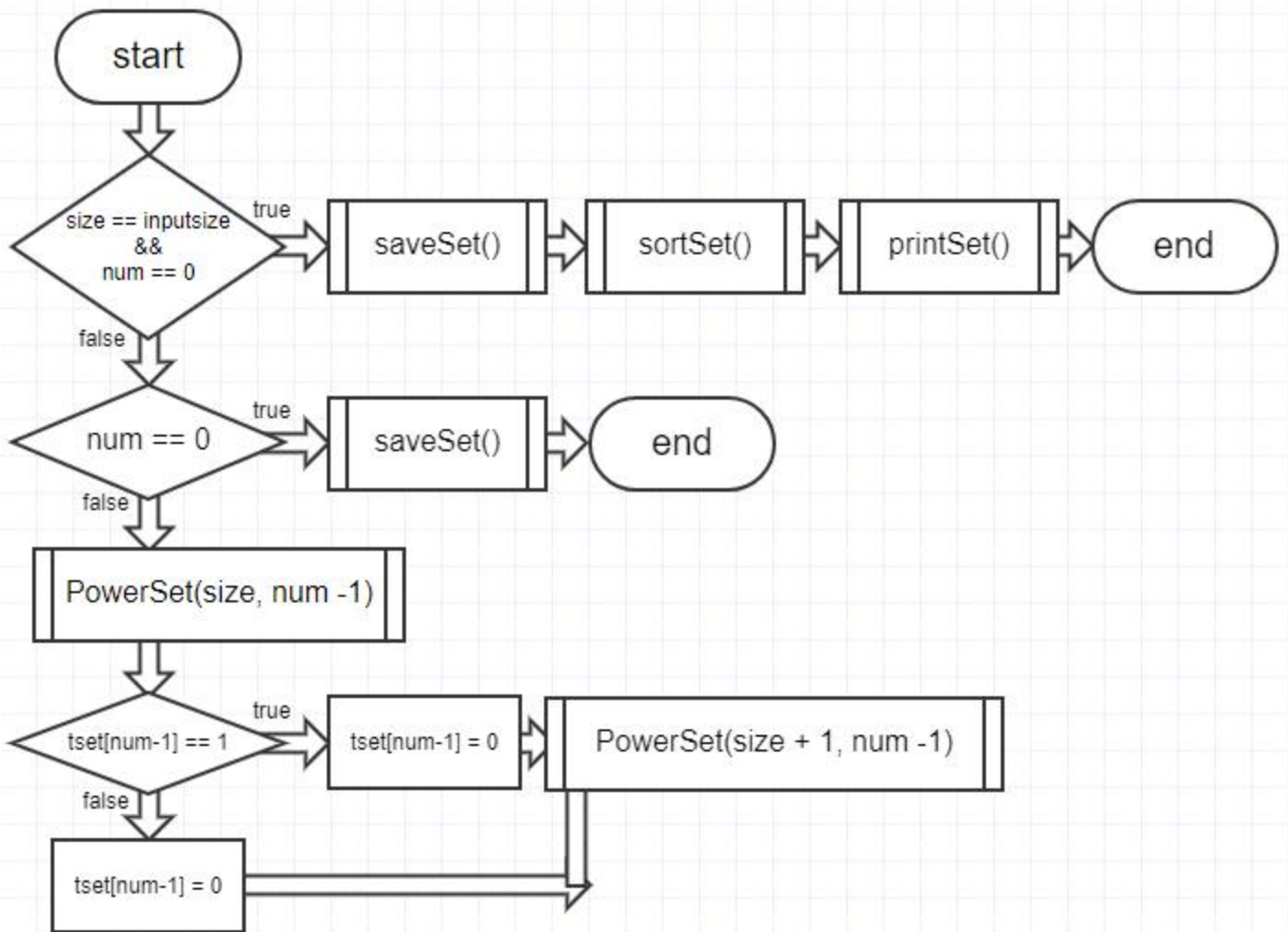
문제1은 숫자 n 을 입력받아 n 번째까지의 소문자 알파벳을 원소로 가지는 멱집합을 모두 구하여 정렬된 상태로 출력하는 문제이다.



1.cpp에서 사용한 global variable과 struct는 위와 같다. inputsize는 문제에서 입력받는 int n 에 해당하며 alpha는 set 출력 시 사용할 순서대로 sort된 소문자 알파벳의 모음이다. tset(temp set)은 PowerSet에서 set에 포함된 element를 표현하기 위해 사용되는 set이며 struct rset(result set)은 구한 모든 멱집합을 저장하고 sort하기 위해 만든 struct이다. 따라서 이 struct를 이용하여 멱집합을 저장하고 sort할 rset result를 10000의 크기로 선언하였다.(멱집합의 개수는 2^n 이기 때문에 크게 선언하였다.) resultnum은 rset result에 set가 하나씩 저장될 때 1씩 증가시켜 result의 개수를 파악하기 위한 int variable로 sort 시에 사용하기 위함이다.

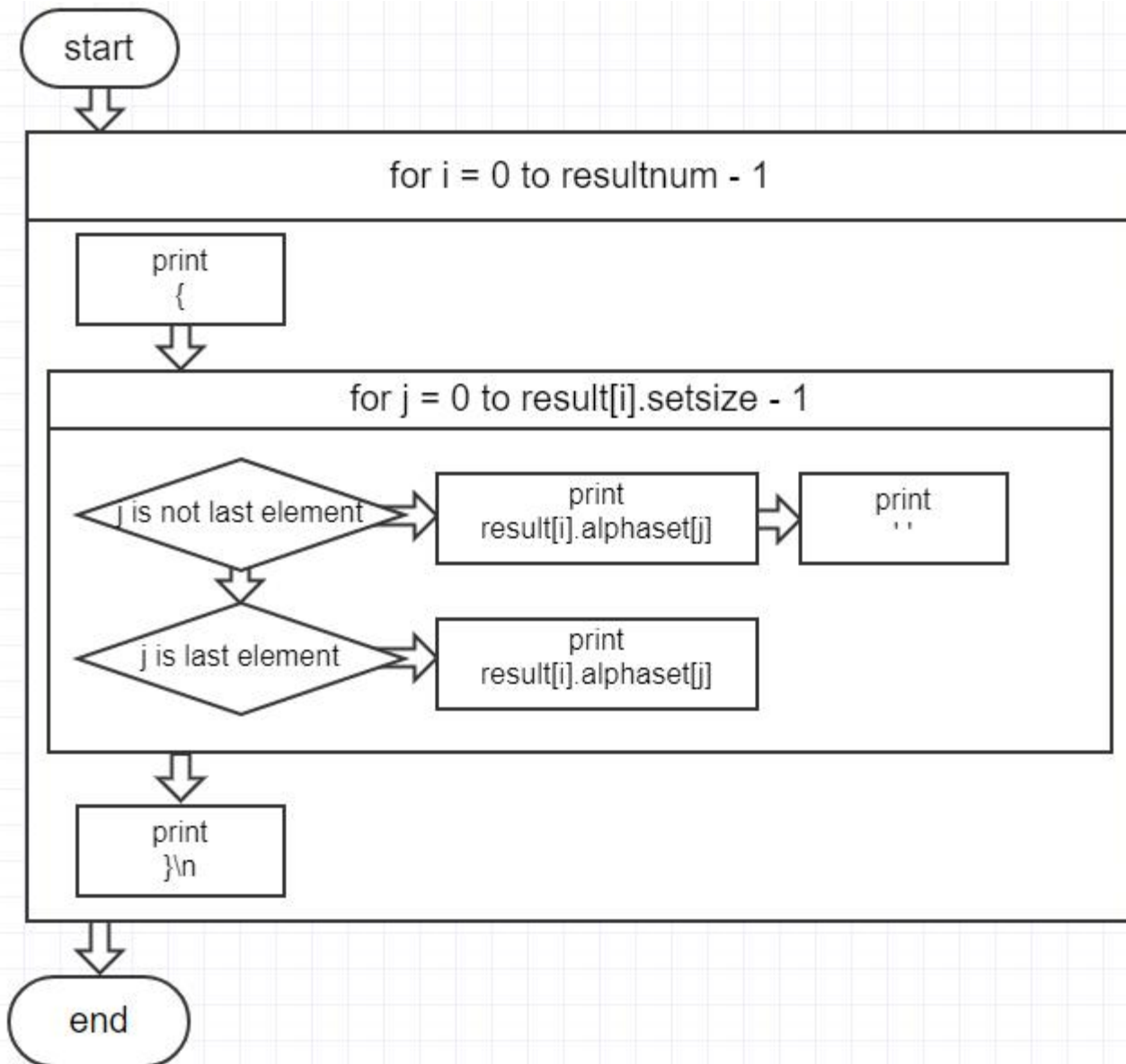


main 함수는 멱집합의 size에 해당하는 inputsize를 입력받아 이를 이용하여 PowerSet call의 argument로 사용한다. 이후 PowerSet function에서 멱집합의 각 부분집합을 구하고 정렬하고 출력하면 프로그램은 끝난다.



void PowerSet(int size, int num)

`PowerSet` function은 첫번째 argument인 `size`와 두번째 argument인 `num`을 각각 증가/감소시키면서 `PowerSet` function을 recursive하게 call한다. `size`를 통해 멱집합의 `size`를 결정하고 `num`을 통해 각 element의 포함 유무를 결정한다. function이 recursive하게 run되다가 `num`이 0이 되면 해당 set을 `rset` result에 save한다. 또한 `num`이 0이 됨과 동시에 `size`가 `inputsized`(입력받은 정수)와 같아지면 set을 save하고 `sortSet()`을 call하여 element들을 sort하고 `printSet()`을 call하여 각 element들을 출력한다.

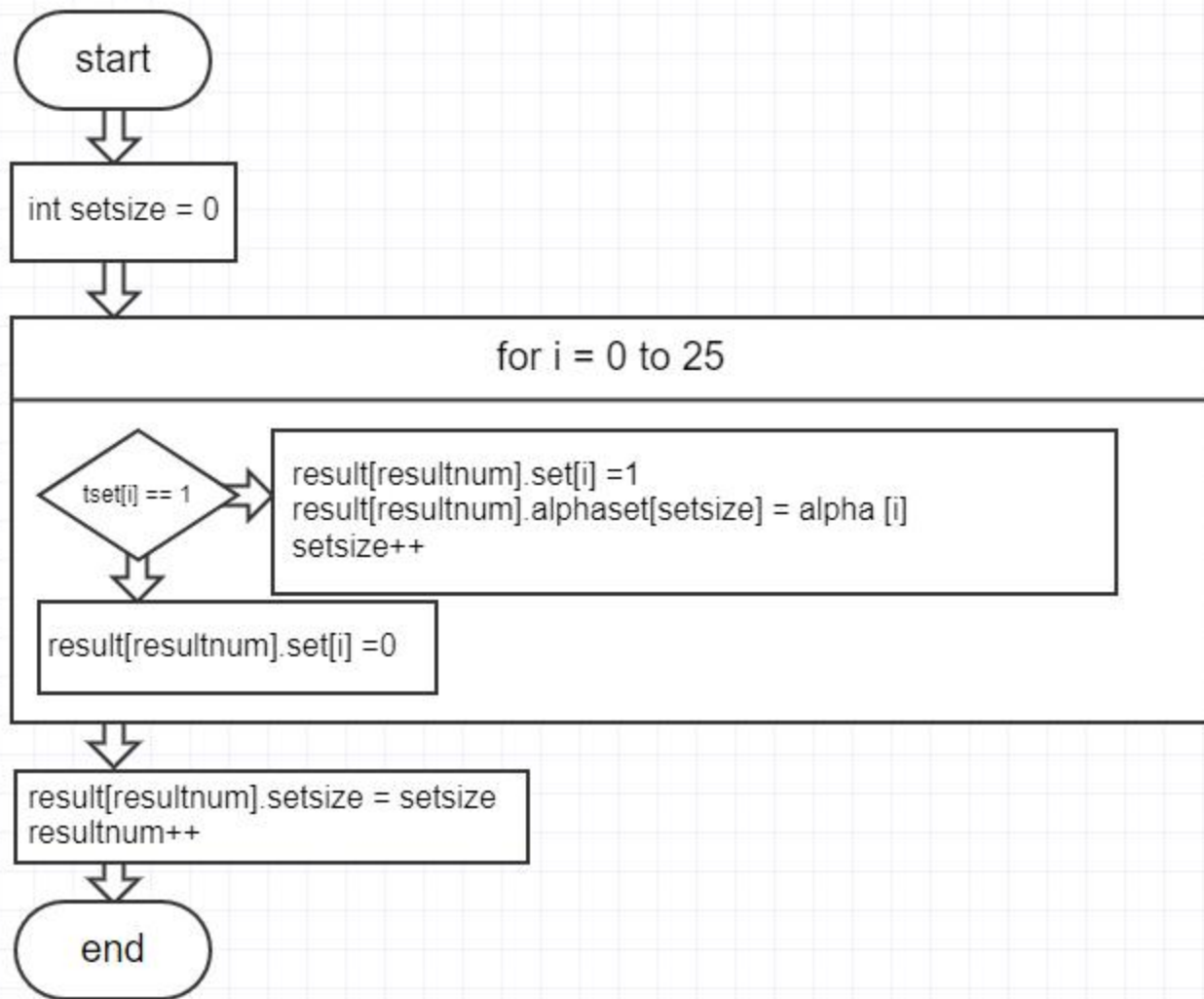


void printSet()

printSet()은 result에 저장된 set들을 모두 출력하는 function이다.

각 set의 맨 처음 element 출력 이전 { 를 출력한다.

result[i].alphaset[j]에 저장된 element가 마지막 element가 아니라면 알파벳을 출력하고 ' ' blank를 하나 출력한다. 이후 element가 마지막 element라면 알파벳만 출력하고 '}'을 출력하여 다음 set를 출력한다.



void saveSet()

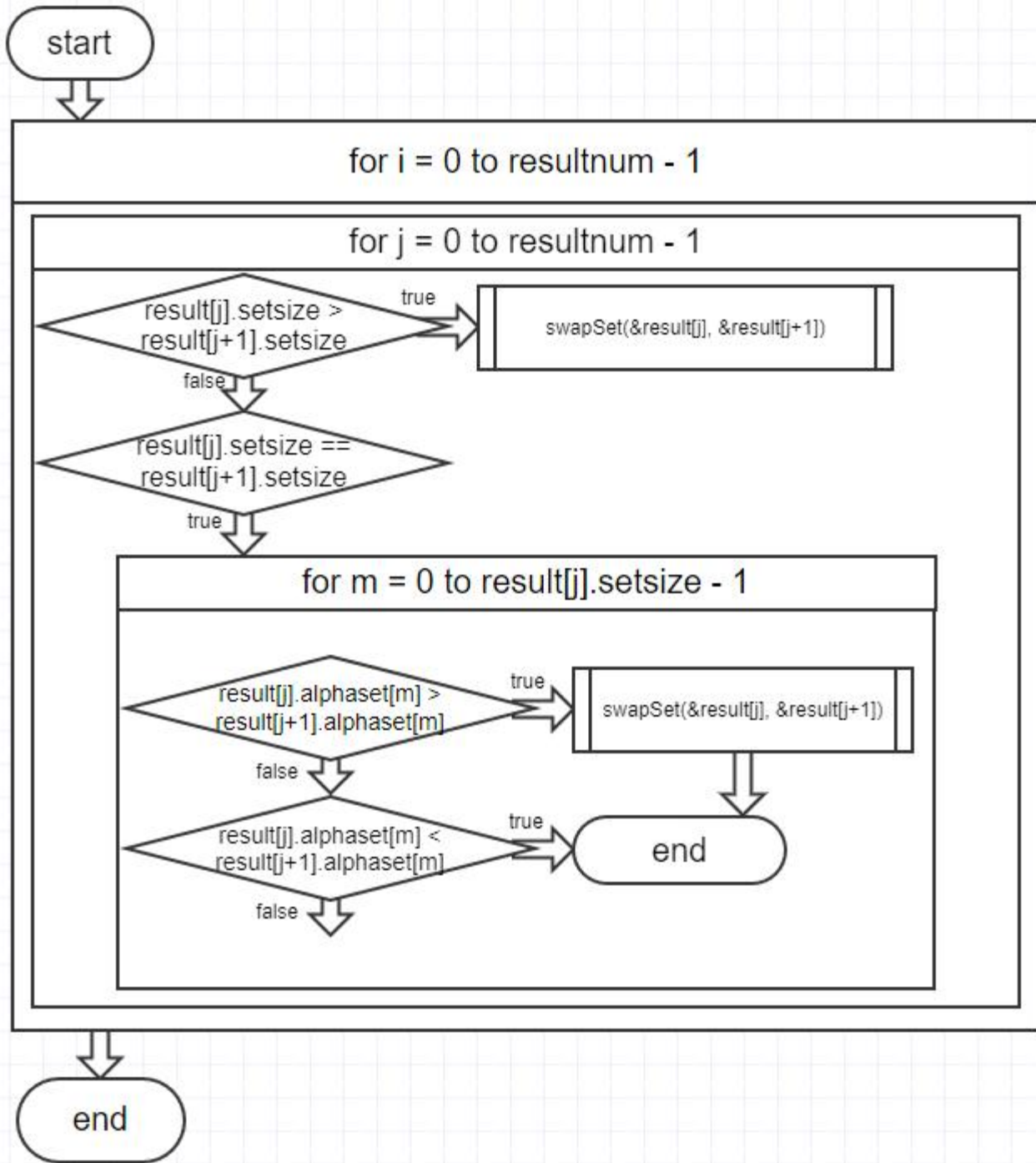
`saveSet()`은 가장 먼저 `int` variable `setsize`를 0으로 initialize하면서 시작한다. 이는 save하는 set의 size를 count하기 위함이다.

이후 `for` loop에서 `tset(tempset)`에 들어있는 element를 읽으면서 1인 element가 있으면 `result`의 member variable인 `set`의 `i`번째 element를 1로 save한다. 그리고 `result`의 또다른 member variable인 `alphaset`에 `i`번째의 알파벳을 save한다. 그리고 `result`에 element를 추가하였으므로 `setsize`를 1 증가시킨다.

`tset`의 `i`번째 element가 0이라면 마찬가지로 `result`의 `set`의 `i`번째 element도 0으로 save해준다.

`tset`의 모든 element를 다 save하고 나면 `tset`의 1의 개수가 count됐으므로 `result`의 member variable인 `setsize`에 count한 `setsize`를 save해준다.

또한 `result`에 하나의 element가 추가되었기 때문에 `resultnum`을 1 증가시켜준다.



void sortSet()

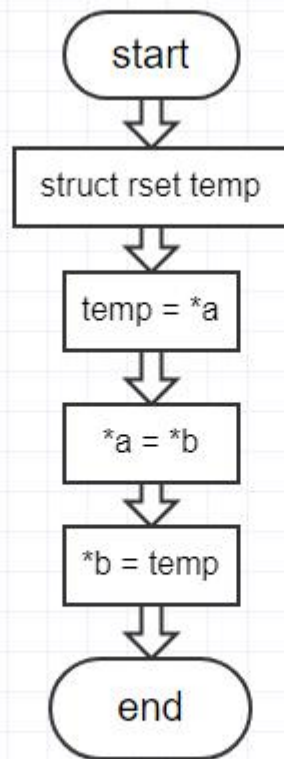
sortSet은 PowerSet에서 save한 멍집합을 element와 알파벳의 순서대로 sort해주는 function이다.

전체적인 구조는 bubble sort의 구조를 따랐으나 단순한 size비교 sorting이 아닌 element size+order의 이중 sorting이기 때문에 내부의 j for loop에서 반복횟수에 i를 빼지 않았다.

따라서 sorting의 flow는 result에 저장된 j번째 element의 setsize를 비교하여 j번째의 setsize가 j+1번째의 setsize보다 크면 swapSet function으로 element를 swap하였다.

반면 j번째의 setsize가 j+1번째의 setsize와 같으면 해당 set들의 안에 있는 element의 알파벳 순서를 보아야한다. 따라서 for loop을 이용하여 j번째의 m번째 alphaset이 j+1번째의 m번째 alphaset보다 크면 swapSet을 call한다.

이와 같은 function으로 구한 멍집합을 sort하였다.



void swapSet(struct rset* a, struct rset* b)

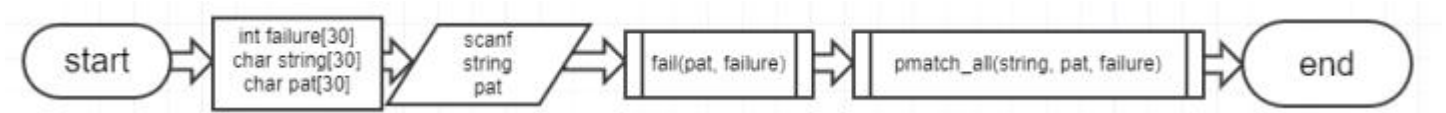
swapSet은 흔히 우리가 사용하는 temp에 a를 저장해두었다가 a에 b를 저장, temp를 a에 저장하는 swap 방식을 사용하였다.

제시된 출력 예시를 만족한다.

```
cse20211562@cspro:~/structure/hw2$ gcc 1.c
cse20211562@cspro:~/structure/hw2$ ./a.out
Insert the size of set: 3
{} {a} {b} {c} {a b} {a c} {b c} {a b c}
```

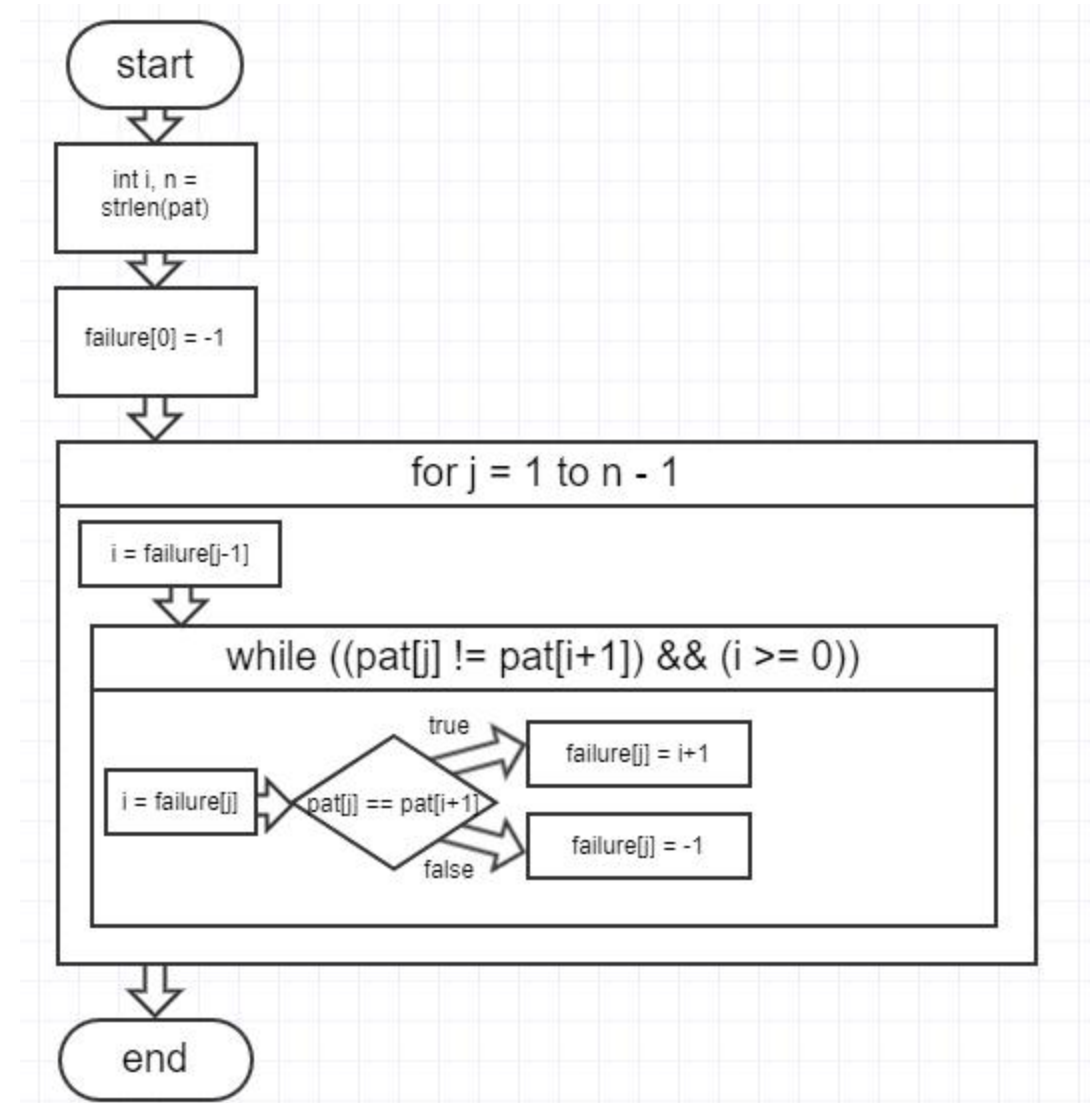
문제2

문제2는 kmp algorithm을 사용하여 string에 pattern과 일치하는 부분이 있는지 탐색하여 일치하는 부분의 string index를 출력하는 pmatch_all() function을 작성하는 문제이다.



int main()

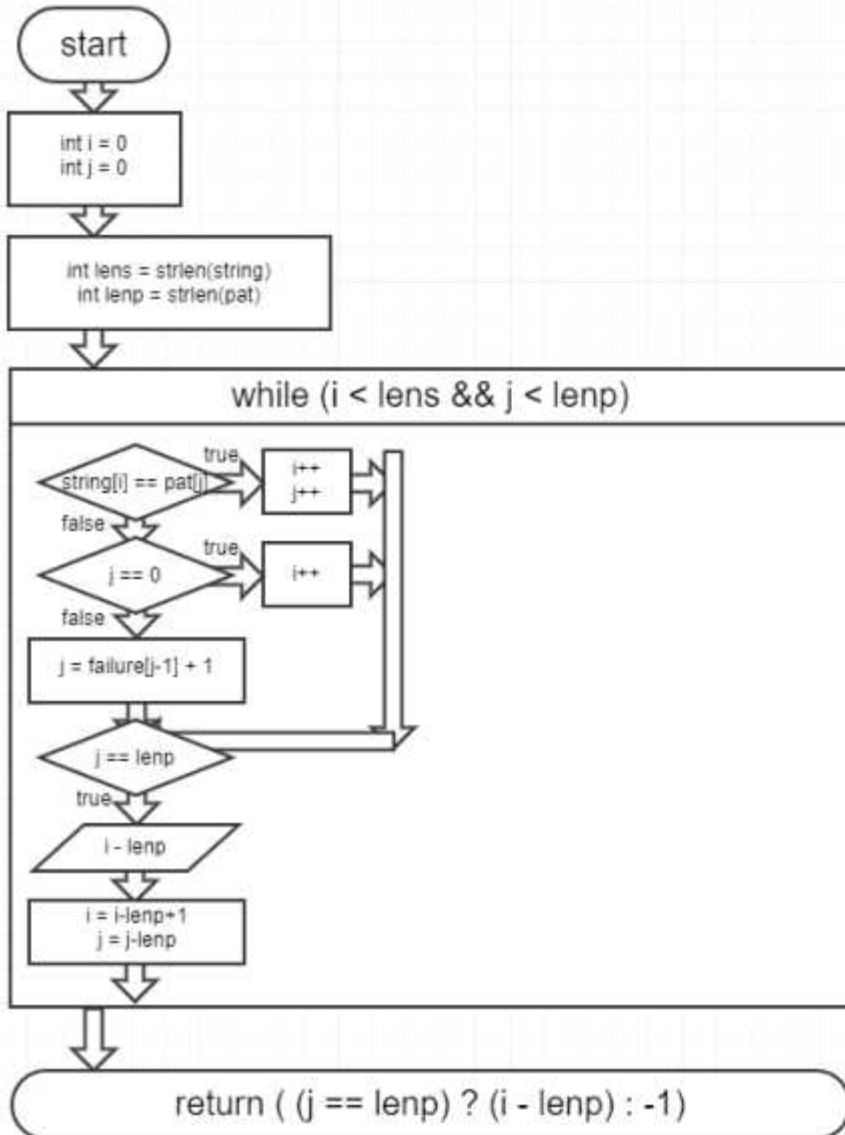
main function은 fail, pmatch_all function call에 필요한 argument인 failure, string, pat을 선언하고 string, pat은 입력받는다. 이후 fail function을 call하고 pmatch_all을 call하여 일치하는 부분의 starting index를 출력한다.



void fail(char *pat, int *failure)

fail function은 kmp algorithm에서 사용하는 fail array를 pattern에 대해서 만들기 위한 function이다. int variable인 i와 n은 pattern의 length를 save하고 failure의 0번째 element는 -1로 save한다. for loop이 돌면서 i가 failure[j-1]과 일치할 경우 i가 음수가 아니고 pat[j]와 pat[i+1]이 다른 동안 repeat하는 while loop에 진입한다.

I를 failure[j]로 save하고 j번째 pattern이 i+1번째 pattern과 일치할 경우 j번째 failure는 i+1을 save, 아닐 경우는 -1을 save한다.



int pmatch_all(char *string, char *pat, int* failure)

pmatch_all function은 while loop에 필요한 int variable i , j는 0으로 initialize하고 int variable lens, lenp는 각각 string과 pat의 length를 save한다.

I가 lens 이하이고 j가 lenp이하인 while loop에서는 i를 순차적으로 증가시키면서 i번째의 string의 element가 pat의 j번째 element와 일치할 경우 동시에 오른쪽으로 한칸씩 비교하면서 중간(j != 0)에 element가 불일치 할 경우 j에 failure[j-1] +1을 save한다.

이후 j가 pattern의 길이와 같아진다면(string에서 pattern과 일치하는 부분을 찾는다면) starting point(i - lenp)를 출력하고 i에는 starting point + 1을 save하여 바로 다음 인덱스부터 다시 search를 하도록 하고 j는 lenp를 빼주어 다시 첫번째 pattern의 element와 비교를 할 수 있도록 해준다.

제시된 출력 예시를 만족한다.

 cse20211562@cspro: ~/structure/hw2

```
1 bbbbbbabbbbbc
2 bbb
```

```
~
~
~
```

```
cse20211562@cspro:~/structure/hw2$ gcc 2.c
cse20211562@cspro:~/structure/hw2$ ./a.out < 2.txt
0
1
2
6
7
8
```

 cse20211562@cspro: ~/structure/hw2

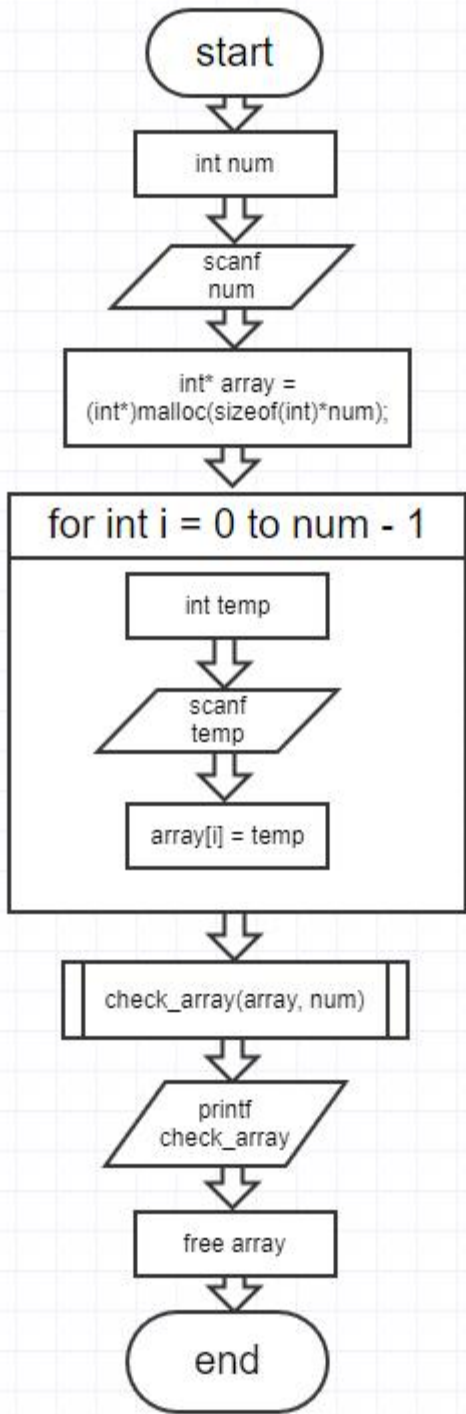
```
1 bbbbbbabbbbbc
2 aa
```

```
~
~
~
~
```

```
cse20211562@cspro:~/structure/hw2$ gcc 2.c
cse20211562@cspro:~/structure/hw2$ ./a.out < 2.txt
cse20211562@cspro:~/structure/hw2$
```

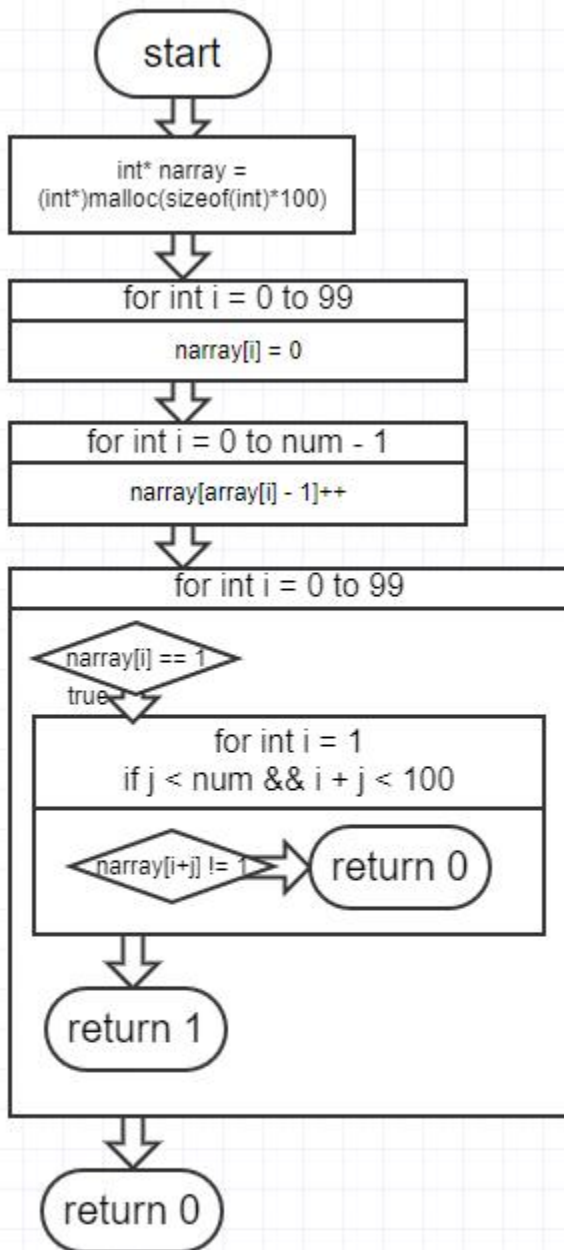
문제3

문제3은 array를 parameter로 받아 element들이 연속된 숫자만 존재하는지 확인하는 check_array() function을 만드는 함수이다. 연속된 숫자만 존재하는 동시에 중복되는 숫자도 존재하지 않아야 한다.



`int main()`

`main` function은 array의 size인 `num`을 입력받아 array를 save할 integer pointer array의 memory allocation을 해준다. 이후 array에 하나씩 element를 저장하여 `check_array`에 array와 `num`을 parameter로 넘겨준다. 이후 확인이 끝나면 그 리턴 값을 출력하고 array의 memory를 free 해준다.



int check_array(int* array, int num)

`check_array` function은 먼저 숫자의 array인 int pointer `narray`를 memory allocation해주고 각 index에 0을 save해준다. 이후 `num`만큼 반복되는 loop에서 `narray`의 (`array`의 `i`번째 element -1)번째 element를 1 증가시킨다. => `narray`에 `array`의 element 값을 1로(존재한다) 해당 번째 index에 save한다.

이후 100번 반복되는 for loop에서 `narray`의 index를 하나하나 보면서 1이 존재하는 경우(연속하는 첫번째 element를 찾은 경우) 그 index부터 하나하나 보면서 연속된 element가 `num` 만큼 저장되어 있는지 확인한다. 아닐경우 0을 return하고 그만큼 있는 경우 1을 리턴한다.

문제에서 $O(n)$ 시간 복잡도를 가지라 하였으므로 `check_array`는 최대 $100 \cdot \text{num}$ 번 실행되므로 $O(n)$ 의 시간 복잡도를 가진다.

제시된 출력 예시를 만족한다.

```
cse20211562@cspro: ~/structure/hw2
```

```
1 5
2 4 1 5 2 3
~
~
```

```
cse20211562@cspro:~/structure/hw2$ gcc 3.c
cse20211562@cspro:~/structure/hw2$ ./a.out < 3.txt
1
cse20211562@cspro:~/structure/hw2$
```

```
cse20211562@cspro: ~/structure/hw2
```

```
1 6
2 10 14 12 15 11 9
~
~
```

```
cse20211562@cspro:~/structure/hw2$ ./a.out < 3.txt
0
cse20211562@cspro:~/structure/hw2$
```

```
cse20211562@cspro: ~/structure/hw2
```

```
1 1
2 1
~
~
```

```
cse20211562@cspro:~/structure/hw2$ ./a.out < 3.txt
1
cse20211562@cspro:~/structure/hw2$
```

```
cse20211562@cspro: ~/structure/hw2
```

```
1 4
2 2 2 3 1
~
~
```

```
cse20211562@cspro:~/structure/hw2$ ./a.out < 3.txt
0
cse20211562@cspro:~/structure/hw2$
```

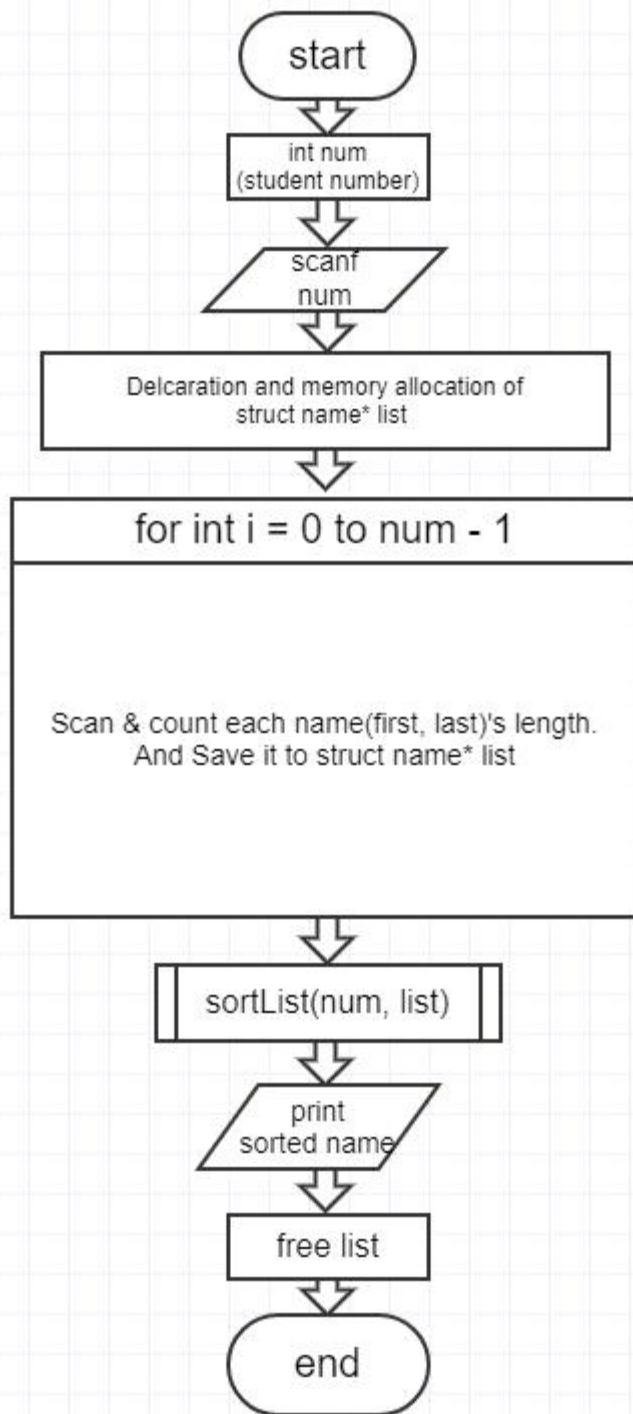
문제4

문제4는 학생 수와 학생 수 만큼의 이름을 입력받아 그들을 정렬하여 출력해야하는 문제이다.

```
struct name{
    char* lname // lastname
    int llen    // length of lastname
    char* fname // firstname
    int flen     // length of firstname
}
```

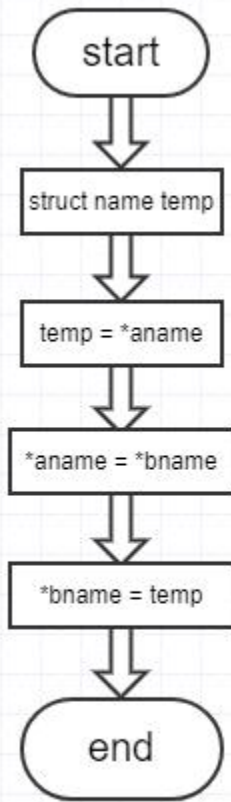
struct name

성과 이름에 대하여 둘 다 sort 해야하는 case이므로 name struct를 declare하여 그 안에 성, 성의 길이, 이름, 이름의 길이를 저장하고자 했다.



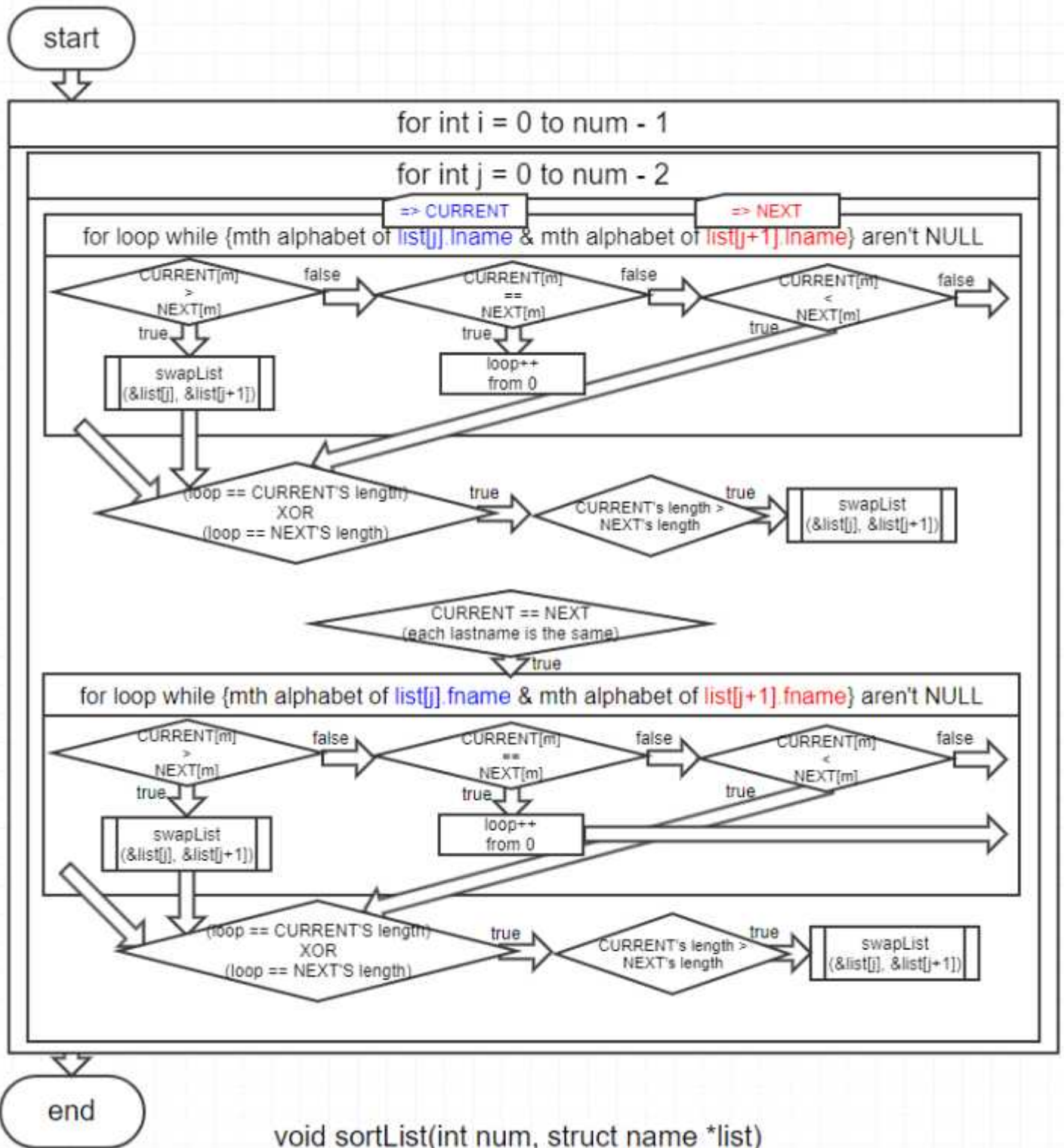
int main()

main function에서는 학생 수를 입력받고 그만큼의 struct name pointer list를 memory allocation한다. 이후 for loop에서 각 이름의 성, 이름의 길이와 char 그 자체를 list에 save 한다. 그리고 parameter로 num과 list를 넣어 sortList를 call 한다. 이후 sort된 list를 순서대로 출력하고 각각의 memory를 free 해준다.



```
void swapList(struct name* aname, struct name* bname)
```

swapList은 흔히 우리가 사용하는 temp에 a를 저장해두었다가 a에 b를 저장, temp를 a에 저장하는 swap 방식을 사용하였다. datatype만 struct name일 뿐이다.



void sortList(int num, struct name *list)

sortList의 전체적인 flow는 set의 size와 각 element의 order에 대해 sort했던 1번 문제와 유사하다.

단, 이 경우는 성의 order와 이름의 order에 대해서 sort하는 문제이기 때문에 일부 flow는 다르다.

이중 loop를 씌워주고 가장 먼저 성에 대해서 j번째와 j+1번째에 대해서 비교를 한다. 각 성의 m번째 element를 비교하면서 CURRENT(j번째)가 NEXT(j+1번째)보다 큰 경우(순서가 안 맞는 경우) swapList를 call하여 swap하고 뒤 element는 더 볼 필요가 없으니 break한다. CURRENT가 NEXT와 같은 알파벳인 경우 loop를 1 증가시켜주고, 알파벳이 작은 경우 순서가 맞는 것이니 for loop에서 break해준다.

loop 이라는 integer variable을 설정한 이유는 while loop이 각 성의 string의 길이가 다른 경우 더 짧은 길이만큼만 비교하게 되어 더 긴 성의 나머지 character는 알 수가 없게 된다. 따라서 성이 cho, choi인 경우 loop는 3이 되니 cho의 길이와는 같고 choi의 길이와는 다르니 XOR연산을 통해 이런 예외를 연산하였다. 따라서 cho choi와 같은 경우는 swap해주고 아예 성이 길이까지 같은 경우에는 이제 이름을 비교해주

어야 한다.

이름의 경우도 똑같이 m번째 element가 CURRENT가 NEXT보다 더 큰 경우 swap해주고 break, 같은 경우는 loop 1 증가, 작은 경우는 break한다. 이후 jin, jini와 같은 경우는 loop과 CURRENT, NEXT의 길이의 XOR 연산으로 CURRENT가 더 길이가 긴 경우 swap해준다.

제시된 출력 예시를 만족한다.

 cse20211562@cspro: ~/structure/hw2

```
1 6
2 Kim Minsu
3 Kim Minju
4 Choi Hojeong
5 Cho Yujin
6 Lee Minsu
7 Choi Minjeong
```

```
cse20211562@cspro:~/structure/hw2$ gcc 4.c
cse20211562@cspro:~/structure/hw2$ ./a.out < student.txt
Cho Yujin
Choi Hojeong
Choi Minjeong
Kim Minju
Kim Minsu
Lee Minsu
cse20211562@cspro:~/structure/hw2$
```